Apache Spark + PySpark

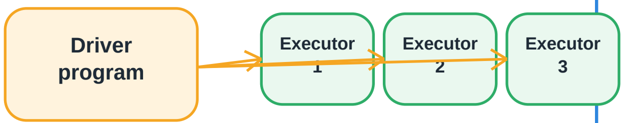
Distributed data processing in plain English - visual written guide

1 What Spark is really doing

Apache Spark is a distributed processing engine. In data engineering, it is used to read, transform, join, aggregate, and write large datasets across a cluster.

Spark = engine for heavy data work
PySpark = Python API for Spark

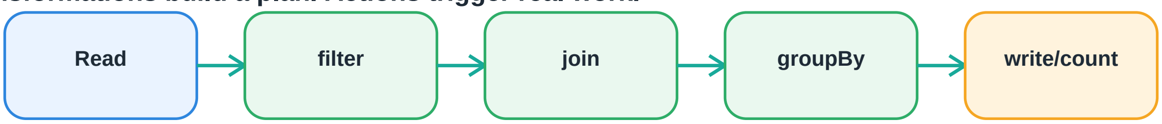
Spark = factory, PySpark = control language



Driver coordinates. Executors do parallel data work.

2 The lazy execution idea

Transformations build a plan. Actions trigger real work.



Nothing really executes until an action runs, such as count(), show(), collect(), or write().

3 Core concepts at a glance

Driver

Runs the main program, builds the plan, coordinates jobs.

Executor

Worker process that runs tasks and stores/caches data.

DataFrame

Distributed table with rows, columns, schema, and transformations.

Partition

A slice of the data processed by tasks in parallel.

Transformation

Lazy operation: select, filter, join, groupBy, withColumn.

Action

Triggers execution: count, show, collect, write.

Spark Architecture

How the pieces work together

Driver

Runs the PySpark application, builds the logical/physical plan, and requests executors.

Cluster Manager

Allocates resources. Can be YARN, Kubernetes, standalone, or cloud-managed service.

Executors

Run tasks in parallel, hold cached data, and report status back to the driver.

Tasks and Partitions

A task works on one partition. Parallelism depends heavily on partition count and data size.

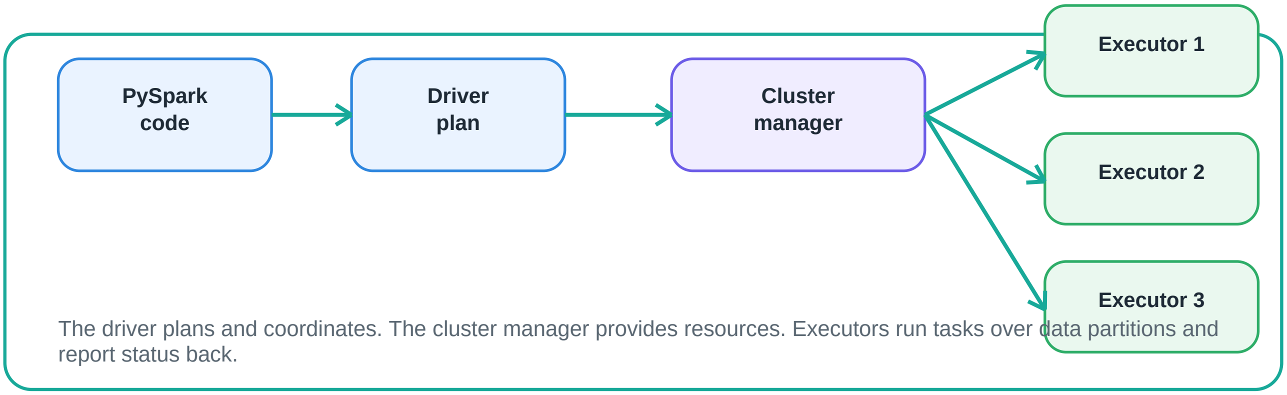
Storage

Input/output may be HDFS, S3, ADLS, tables, Parquet, CSV, JSON, or warehouse sources.

Spark UI / Logs

Shows jobs, stages, tasks, shuffle, skew symptoms, retries, and failed executors.

5 Architecture data flow

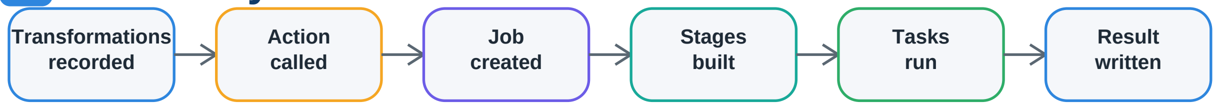


The driver plans and coordinates. The cluster manager provides resources. Executors run tasks over data partitions and report status back.

Running PySpark Jobs

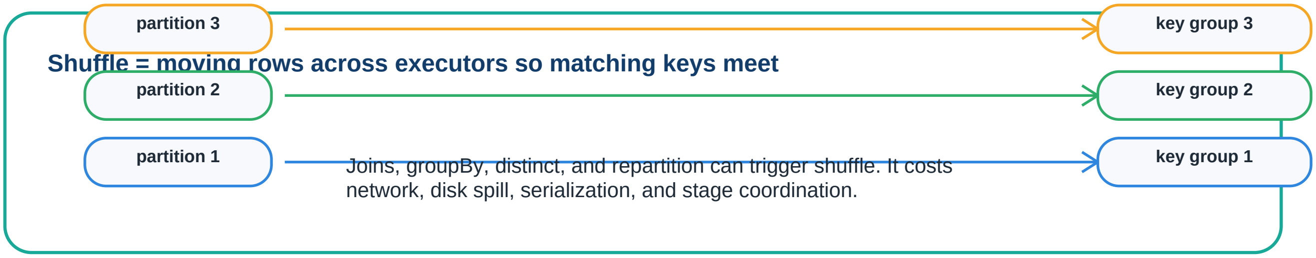
Lazy plans, stages, shuffle, and code pattern

6 Job lifecycle



The driver owns the plan. Actions create jobs. Jobs split into stages. Stages split into parallel tasks over partitions.

7 Why shuffle matters



8 Minimal PySpark ETL pattern

A simple read-transform-write pattern

```
from pyspark.sql import functions as F

clean_df = (
    telemetry_df
    .filter(F.col("event_time").isNotNull())
    .join(reference_df, on="server_id", how="left")
    .groupBy("application", "service", "process_date")
    .agg(
        F.count("*").alias("row_count"),
        F.avg("metric_value").alias("avg_metric"),
        F.max("metric_value").alias("peak_metric")
    )
)

clean_df.write.mode("overwrite") \
    .partitionBy("process_date") \
    .parquet(output_path)
```

How to Think About Spark

Use cases, boundaries, production habits, and interview language

10 Spark is / Spark is not

Spark is

- A distributed data processing engine
- A way to process large batch datasets
- A DataFrame and SQL processing platform
- Useful for ETL, joins, aggregation, ML features

Spark is not

- A workflow scheduler by itself
- A magic fix for bad data logic
- Always faster for small data
- A replacement for data quality checks

11 Best use cases

ETL / ELT

Clean, join, aggregate, and write curated data.

Telemetry

Group metrics by host, app, service, and time.

Warehouse prep

Build fact-like or reporting-ready datasets.

Forecast inputs

Prepare time-series features and summaries.

Data quality

Row counts, schema checks, null and duplicate checks.

12 Production habits that matter

- Prefer explicit schemas for production reads.
- Validate row counts before and after major transformations.
- Watch joins, groupBy, distinct, repartition, and shuffle.
- Use broadcast joins only when the lookup side is small enough.
- Partition outputs by useful query or process keys, such as process_date.
- Do not use collect() on large datasets; avoid repeated count() unless needed.
- Use Spark UI/logs to inspect failed stages, task retries, spills, and skew.

13 Interview-ready explanation

Spark is the distributed engine that processes data across executors. PySpark is the Python API I use to build DataFrame pipelines. The key ideas are lazy transformations, actions that trigger jobs, stages and tasks that run across partitions, and shuffle as the expensive operation to watch.